

A Decision-Focused Surrogate for AC Unit Commitment: Learning the Linearisation Point and a Cardinality Cut

1 Problem

AC unit commitment (AC-UC) selects which generators run, $u \in \{0, 1\}^G$, and their output $p^g \in \mathbb{R}^G$, subject to the AC power-flow equations. Because bus voltages enter through the products $V_i V_j$, the problem is a nonconvex MINLP. On **case118** (118 buses, 54 generators, 186 branches) an exact spatial branch-and-bound solve takes 600 s.

The QCAC relaxation. Constante-Flores and Li replace each voltage product with its first-order expansion about a point $(V_0^{\text{re}}, V_0^{\text{im}})$ and absorb the residual in a slack $\xi \geq 0$ priced at ρ . Writing the lifted variables c_{ii}, c_{ij}, s_{ij} for $|V_i|^2, \text{Re}(V_i V_j^*), \text{Im}(V_i V_j^*)$,

$$|c_{ii} - (2V_{0,i}^{\text{re}} v_i^{\text{re}} + 2V_{0,i}^{\text{im}} v_i^{\text{im}} - |V_{0,i}|^2)| \leq \xi_i, \quad (1)$$

and analogously for c_{ij}, s_{ij} . The expansion is accurate only near V_0 , so the published method *iterates*: solve, re-linearise at the solution, escalate ρ , repeat. On **case118** this converges in 3–4 solves, each carrying all 54 binaries, and reproduces the global optimum (164,101.4, 14 units committed).

Goal. Replace that loop with *one continuous solve*.

2 Method

A network reads the instance parameters $x = (p^d, q^d)$ and emits two objects; the solver then runs once.

$$x \longrightarrow \underbrace{(V^{\text{re}}, V^{\text{im}})}_{\text{linearisation point}}, \quad \underbrace{[n_{\text{lo}}, n_{\text{hi}}]}_{\text{cardinality cut}} \quad (2)$$

Head 1: the linearisation point. QCAC iterates *in order to find* this point, so predicting it removes the iteration. Its value is not assumed: at the exact point, one continuous solve plus rounding recovers the optimal commitment on 8/8 instances (−0.002% gap, 100% agreement) – *while 39.9 of 54 generators are still fractional*. Rounding is therefore not the bottleneck; the linearisation point is.

Head 2: the cardinality cut. The predicted point is never exact, and that error is what costs money. The cut absorbs it:

$$n_{\text{lo}} \leq \sum_i u_i \leq n_{\text{hi}}, \quad n_{\bullet} = n_{\text{min}} + \Delta_{\bullet}(x), \quad (3)$$

anchored on n_{min} , the fewest generators that can meet the reserve (largest capacity first). n_{min} needs no labels, costs a sort and a cumulative sum, and is a *valid* lower bound: the reserve cannot be served with fewer. Measured over 112 instances, $n^* - n_{\text{min}} \in [+6, +9]$ with correlation 0.926, so the head predicts one small, well-conditioned offset.

Algorithm 1 Deployment: one continuous solve

- 1: $(V, n_{\text{lo}}, n_{\text{hi}}) \leftarrow f_{\theta}(p^d, q^d)$
 - 2: solve the QCAC relaxation linearised at V , with (3) hard and $u \in [0, 1]^G$ $\triangleright$ no binaries
 - 3: $z \leftarrow \mathbf{1}[u > 0.5]$
 - 4: **reserve top-up:** switch on by descending u until $p^{\max\top} z \geq (1 + r) \sum p^d$
 - 5: **upward repair:** while infeasible, commit the next-highest u
 - 6: **downward repair:** for the least-confident committed units, try switching off; keep if feasible and cheaper
 - 7: **return** z and its AC dispatch
-

The cut must be hard. This is the single most important implementation detail. Imposed *softly* – as a penalty $\lambda\sigma$ on the violation σ – the cut is inert whenever it binds: if it is violated at the optimum, the penalty gradient in u is the constant λ and the right-hand side n_{hi} *drops out of the optimality conditions entirely*. Empirically, replacing the learned band with $[0, 0]$, or with $[0, 0]$ shifted by -20 , reproduced its results bit-for-bit. We impose (3) as a hard constraint, with a softly-priced fallback if that renders an instance infeasible.

Training. Self-supervised on the deployment cost of Algorithm 1; no optimal commitment enters the loss. Rounding makes that cost piecewise constant in the network outputs, so there is no pathwise derivative; we use an antithetic Gaussian smoothing estimator

$$\widehat{\nabla}_y J = \frac{1}{P} \sum_{p=1}^P \frac{J(y + \Sigma \epsilon_p) - J(y - \Sigma \epsilon_p)}{2} \Sigma^{-1} \epsilon_p, \quad \epsilon_p \sim \mathcal{N}(0, I), \quad (4)$$

with a *per-group* Σ : the voltage outputs are $O(10^{-1})$ while the cardinality bounds are measured in generators, $O(10)$. A common σ perturbs the bound by a fraction of a unit, which can never change a rounded commitment – those coordinates then receive exactly zero signal.

3 Implementation

The relaxation is an SOCP; do not give it to a MINLP solver. Gurobi solves the *continuous* QCAC relaxation – which has no integer variables – by spatial branch-and-bound, because it will not certify $c_{ij}^2 + s_{ij}^2 \leq c_{ii}c_{jj}$ as a rotated cone (8 of **case118**’s 186 remain bilinear). It reports “*continuous model is non-convex – solving as a MIP*” and hits a 120s limit. The same problem is an SOCP: CLARABEL solves it in 0.077s, returning the same cost to the digit at every $\rho \in [10^2, 10^6]$ – a 1200–2000 $\times$ difference, and the difference between a trainable method and an untrainable one. Gurobi is retained only where binaries genuinely exist (the reference solver).

The rank-1 cut is load-bearing. Dropping $c_{ij}^2 + s_{ij}^2 \leq c_{ii}c_{jj}$ makes the solve 158 $\times$ faster and the slack still converges to 10^{-10} , but the solution leaves the rank-1 manifold: **case118** then returns 161,352.6 against a proven optimum of 164,101.4 – a cost *below* the optimum, i.e. not physical. At zero slack the linearisation only forces c_{ii} to equal the first-order expansion of $|V_i|^2$, not $|V_i|^2$ itself.

Instances. One *correlated* system-level draw per instance, $s \sim U[0.80, 1.15]$, with small per-bus jitter. An i.i.d. per-load factor – the obvious choice – averages out the aggregate variation that the reserve constraint and merit order respond to: it gives under 1% demand variation on **case118** and

1–2 distinct optimal commitments out of 32. The correlated draw gives $\sim 10\%$ variation and 46 distinct commitments over 192 instances (mean 14.7 units committed).

Setup. 192 instances, 144 train / 48 held out. References from QCAC iterative (158s each on average). The network is a two-layer SiLU trunk (256 wide) with a voltage head and a cardinality head; $v_{\text{scale}} = 0.5$. Training and evaluation use the *same* restoration depth — a mismatch causes the network to compensate for a repair asymmetry that is absent at evaluation time.

4 Results

Both cases run an *identical* configuration: $K=1$ cardinality cut imposed hard, 45 epochs, full batch, the same restoration depth in training and evaluation, the same learning rate and perturbation scales, and a band initialisation derived from $n_{\min}$ rather than chosen per case. Earlier runs differed between the two grids, which makes any cross-case statement a tuning artifact; nothing below does.

Table 1: Held-out instances, one continuous solve each. Identical restoration and AC pricing for every arm — only the source of the commitment differs. *Discrete* is the fraction of generators committed wrongly, *continuous* is $\|p^g - p^{g*}\|_1 / \sum p^{g*}$ on the recovered dispatch, *gap* is true AC cost against the QCAC-iterative reference.

case118 — 192 instances, 144 train / 48 held out				
	labels	discrete	continuous	gap
relaxation only (flat V , no cut)	—	40.32%	85.97%	+10.378%
NN proxy, supervised on u^*	yes	3.32%	5.21%	+0.057%
NN proxy, self-supervised	no	9.61%	12.61%	+0.242%
ours: V only	no	22.53%	23.34%	+3.319%
ours: V + constant penalty	no	4.17%	5.29%	+0.103%
ours: V + learned cardinality cut	no	4.05%	5.92%	+0.049%
<i>control</i> : dummy band	no	28.32%	51.50%	+6.718%
<i>control</i> : random band	no	38.43%	62.14%	+8.933%
<i>control</i> : mean band (no per-instance information)	no	5.63%	10.57%	+0.582%
case300 — 151 instances, 111 train / 40 held out				
relaxation only (flat V , no cut)	—	26.52%	22.73%	+2.154%
NN proxy, supervised on u^*	yes	1.63%	1.27%	−0.006%
ours: V only	no	24.57%	18.78%	+1.861%
ours: V + constant penalty	no	12.79%	9.10%	+0.592%
ours: V + learned cardinality cut	no	8.73%	5.49%	+0.296%
<i>control</i> : dummy band	no	12.46%	7.49%	+0.626%
<i>control</i> : random band	no	15.04%	8.40%	+1.466%
<i>control</i> : mean band (no per-instance information)	no	9.60%	6.85%	+0.605%

Headline. One continuous solve reaches a +0.049% gap on **case118** with 4.05% of generators mis-committed, against +10.378% and 40.32% for the same solve without the learned inputs — $210\times$ on gap, $10\times$ on discrete error. It also beats the *supervised* NN proxy (+0.057%), which trains on optimal commitments this method never sees, and the self-supervised proxy by $4.9\times$. On **case300** the method reaches +0.296% from a +2.154% relaxation, and is $2\times$ better than a constant commitment penalty on all three measures.

The cut is learned, not a disguised constant. Three controls, each designed to refute the claim: a *dummy* band (any always-violated value), a *random* band, and a *mean* band — the learned bounds averaged over the test set, carrying no per-instance information. All fail on both cases; the mean-band control, the sharpest of the three, loses by $12\times$ on **case118** and $2.0\times$ on **case300**. The learned bounds vary across instances with standard deviation 2.28 and 2.19 generators respectively; a constant band has spread 0.

Both heads are necessary. V alone gives +3.319% and +1.861%. The cut alone cannot help, because without a good expansion point the relaxation is solved in the wrong place.

What did not work. Three alternatives were measured and rejected rather than tuned. *Tiering*: splitting the fleet into K merit-order tiers and constraining each is worse at every setting — $K=3$ interleaved +0.351%, random partition +0.807%, contiguous merit blocks +1.757%, against $K=1$'s +0.049%. *A learned rounding threshold* (L2O-MINLP's correction layer): a per-instance threshold ties the fixed 0.5 exactly on **case118**, and the learned head moved to 0.499–0.500 on both cases, scoring identically to fixed rounding. *Confidence-based variable fixing*: helps only with a labelled 96.5%-accurate predictor; with the label-free predictor (9.6% error) it worsens discrete error from 4.17% to 7.68% and makes 7 of 48 instances infeasible.

case118: 48 held-out instances, ONE continuous solve + restoration | reference: QCAC iterative

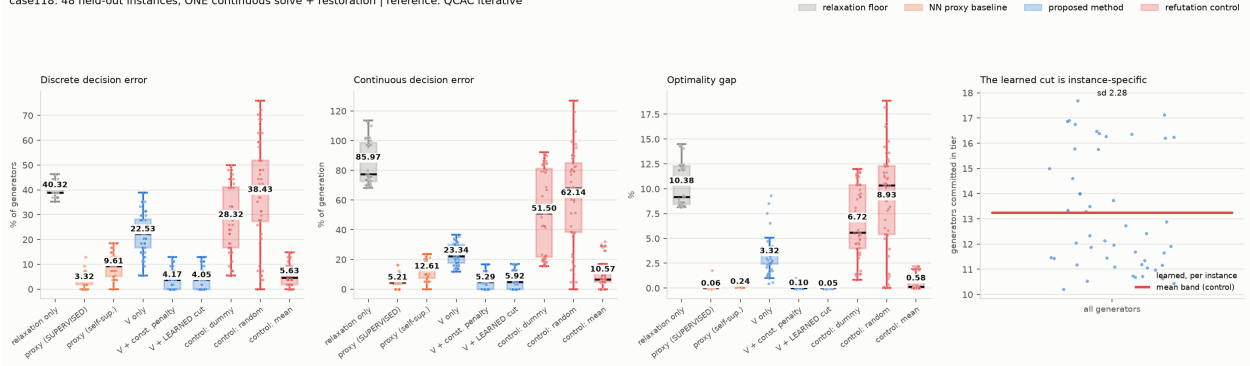


Figure 1: **case118**. The fourth panel is the claim itself: learned bounds scattered per instance rather than collapsing onto the constant (red) that the mean-band control uses.

case300: 40 held-out instances, ONE continuous solve + restoration | reference: QCAC iterative

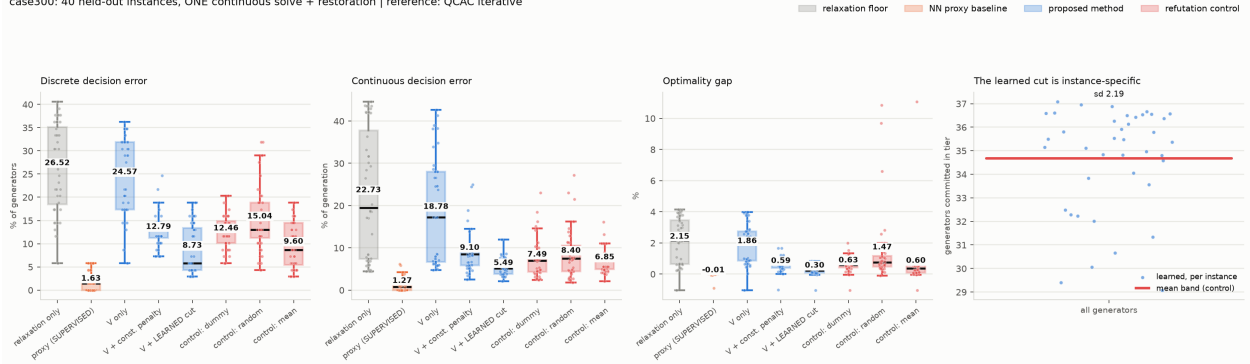


Figure 2: **case300**, same configuration. The supervised proxy is stronger here; the learned cut still doubles the constant penalty on every measure.

5 Limitations

Results are stated relative to QCAC iterative, which reproduces the global optimum where both were run but is not certified optimal everywhere — in one experiment this method beat it on 13 of 24 instances.

case118 is at its ceiling: the oracle linearisation point, which no method can beat, scores -0.002% against this method’s $+0.049\%$. Further gains there are not available.

case300 is data-limited rather than method-limited. Its V head predicts a 600-dimensional correction from 111 training instances and barely improves on the baseline alone ($+1.861\%$ versus $+2.154\%$); the supervised proxy, which sees optimal commitments, reaches -0.006% there. The price of those labels is one reference solve per training instance (≈ 5 h for 111), which this method does not pay.

Seed replication used re-splits of one instance pool, which is closer to cross-validation than to independent replication.